Assignment 8 (Week 8): Searching & the Elementary Sorts

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

October 15, 2026

Concepts That Click

Due: two weeks from issue. There is no automatic late penalty — if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus). Answer every question. Show all working for Numerical and Design questions. Each problem states its own assumptions; everything you need is in Week 8's Notes chapter alone. Notation matches the lecture exactly: `search(A, n, key)` returns an index i with `A[i] == key`, or -1 on a miss; arrays are 0-based and written $A[0..n-1]$; `lo`, `hi`, `mid` are the current search bounds; **sorted** means nondecreasing, $A[0] \le A[1] \le \cdots \le A[n-1]$; a sort is **stable** if equal keys keep their original relative order and **in-place** if it uses $O(1)$ auxiliary space; counting sort assumes keys lie in a known range $0..k-1$. For every bound, name the operation, give the bound, and state the case (Week 2 shape).

8.1 Conceptual

Q1. A search is either a walk or a halving, and which one you get was decided the moment the data was ordered (Karumanchi Ch. 11, pp.547–589 PDF; Wirth Sec. 1.9.1, p. 34; Sec. 1.9.2, pp.34–36) [2, 4].

(a) State the contract of `search(A, n, key)`: what it returns on a hit and on a miss. Then name the one assumption **linear search** makes and the precondition **binary search** *requires*. In one sentence, say precisely what binary search does when that precondition is *violated* — does it crash, or return a plausible wrong answer?

(b) Give linear search's best-, worst-, and average-case comparison counts, naming the case each time. Then explain in one sentence why no additional assumption about *order* can lower its worst case.

(c) Write the worst-case recurrence $T(n)$ for binary search on a range of size n, stating its base case, and unroll it to show the result is $\Theta(\log n)$. Name the case, and contrast it with linear search's worst case.

Why this matters: the precondition is the whole story of this week — this question makes you state it as a contract, not a hint, so that "sort first" becomes a decision and not an afterthought.

Q2. This week's vocabulary decides real choices: two sorts can produce the *same* sorted keys and still be different algorithms (Karumanchi Ch. 10, pp.504–546 PDF; Sedgewick Sec. 2.1, p. 244) [2, 3].

(a) Define **stable** and **in-place**. For each of insertion, selection, bubble, and counting sort, state whether it is stable and whether it is in-place (a four-row table is fine).

(b) A file holds the records $[(P,4),\ (Q,2),\ (R,4)]$, where the label letter is a tie-breaker and the number is the sort key. Give the output a *stable* sort produces and one output an *unstable* sort may produce, then name which of the four sorts above could produce the unstable order.

(c) For each workload, name the single elementary sort that fits best and justify it in one sentence: (i) an array that is already almost sorted; (ii) n keys drawn from a small fixed range $0..k-1$ with $k \ll n$; (iii) an array whose elements are huge records, where moving one element costs far more than comparing two.

Why this matters: "which sort?" is a workload question, not a ranking — this question forces you to read the input's structure and state the choice, which is the skill the week actually tests.

8.2 Numerical

Q3. Trace the two searches on concrete inputs, then use the monotone-predicate pattern the lecture builds for the integer square root (Karumanchi Ch. 11, pp.547–589 PDF; Wirth Sec. 1.9.1, p. 34; Sec. 1.9.2, pp.34–36) [2, 4].

(a) The unsorted array is `[11, 4, 17, 9, 2, 14, 6]` ($n = 7$). Run `lsearch` for `key = 14`: give the comparison count and the returned value. Then run it for `key = 8`: give the comparison count and the returned value.

(b) The array `[2, 5, 8, 11, 14, 17, 20, 23]` ($n = 8$) is sorted, so `bsearch` is admissible. Trace `bsearch` for `key = 5`: write `lo`, `hi`, and `mid` at every iteration, give the comparison count, and give the returned value. Then do the same trace for `key = 13` (absent).

(c) Using the lecture's `isqrt` (binary search on the predicate $P(k) = (k^2 \le n)$ over the answer space $0..n$), compute $\lfloor\sqrt{40}\rfloor$ with integer arithmetic only. Tabulate $P(k)$ for $k = 0, \ldots, 7$, give the true→false boundary, and give the returned value.

Why this matters: a trace is the proof that you know *which* index each comparison lands on — the same discipline that catches a precondition violation before it returns a wrong answer.

Q4. Two sorts on inputs the lecture never traced (Karumanchi Ch. 10, pp.504–546 PDF; Wirth Sec. 2.2.1, p. 46) [2, 4].

(a) Counting sort the array $A = [3, 0, 2, 3, 1, 0, 3, 2]$ with keys in $0..3$. Give k, the tally row (`count` after counting), the prefix-sum row, and the final sorted array produced by the reverse scatter. In one sentence, say which property the reverse scatter protects and why it is needed.

(b) Insertion sort the array `[6, 3, 8, 2, 7, 1]`. After *each* outer iteration $i = 1, \ldots, 5$, write the array, and give the number of shifts performed in that iteration. Finish with the total number of shifts.

Why this matters: counting sort is linear only when you respect its range, and insertion sort is adaptive only when you can see *which* element moved — both facts surface only in a trace.

8.3 Design

Q5. The lecture's `lowerBound` finds the first index with $A[i] \ge$ `key` (the upper bound is the same routine with $>$, Karumanchi Ch. 11, pp.547–589 PDF) [2]. Design the matching `upperBound` and use the pair to count duplicates — a lookup the calculator's symbol table needs when it reports repeated declarations.

Concepts That Click

(a) Write pseudocode for `upperBound(A, n, key)`: the first index i with $A[i] >$ `key` (and n if every element is $\leq$ `key`). Use the same half-open range [`lo`,`hi`) and candidate-retention structure as `lowerBound`; state the one comparison that changes from `lowerBound`.

(b) Explain why the number of occurrences of `key` in a sorted array is exactly `upperBound` − `lowerBound`, and why both routines run in $O(\log n)$. Reference the symbol-table use: a sorted identifier table can report how many times an identifier was declared without scanning it.

(c) Trace `lowerBound` and `upperBound` for `key = 3` on the sorted array $[1, 3, 3, 3, 5, 7, 7, 9]$ ($n = 8$): give `lo`, `hi`, `mid` at each iteration of each routine, the two returned indices, and the occurrence count.

Why this matters: the plain binary search returns *some* match; keeping `mid` as a candidate is what turns it into a boundary finder — this question extends that one idea to both boundaries and back into the running calculator.

Q6. Sorting does not only feed search: putting equal keys next to each other is how we find duplicates (Karumanchi Ch. 10, pp.504–546 PDF; Horowitz & Sahni Ch. 7, standard treatment) [2, 1]. The tokenizer wants a report of identifier *lengths* that occur more than once.

(a) Design `reportRepeats(A, n, k)` for keys in $0..k-1$ using counting sort's tally phase: pseudocode, then the worst-case time and space bounds with their variables. In one sentence, explain why it never compares two keys.

(b) Trace `reportRepeats` on $A = [2, 0, 2, 1, 3, 0, 2]$ (keys in 0..3): give k, the tally row, and the list of values that occur more than once.

(c) The tokenizer also wants those identifiers *sorted by length, keeping their first-appearance order within a length.* Explain why a stable scatter is required, state whether counting sort is in-place, and give its space bound.

Why this matters: the same tally that makes counting sort linear also answers the duplicates question for free — this question designs the reuse and prices the stability it depends on.

References

[1] Ellis Horowitz and Sartaj Sahni. *Fundamentals of Data Structures.* Galgotia Booksource, 2nd edition, 2008.

[2] Narasimha Karumanchi. *Data Structures And Algorithms Made Easy.* CareerMonk Publications, 2017. This calibre-generated edition carries no printed folio numbers; page numbers cited in CS301 material are PDF page indices. NOT a source for Week 1's C-specific pointer/malloc content — see this course's supporting-books index for the coverage check.

[3] Robert Sedgewick and Kevin Wayne. *Algorithms.* Addison-Wesley, 4th edition, 2011.

[4] Niklaus Wirth. *Algorithms and Data Structures.* Author (freely distributed), 2004. Oberon version, August 2004; revision of the 1985 Prentice-Hall edition. Page numbers cited in CS301 material refer to this Oberon PDF, not to the 1985 print edition.

Concepts That Click